МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка

ЗВІТ

до лабораторної роботи №6
з дисципліни «Професійний Python»

Тема: «Автоматизоване тестування Python-програм»

Виконав:
студент Левандівський Назар

Перевірив:
Куньо Іван Михайлович

Львів – 2026

Мета роботи

Сформувати навички професійного автоматизованого тестування Python-програм засобами pytest: unit та integration tests, fixtures, параметризація, перевірка винятків, ізоляція файлової системи й середовища (tmp_path, monkeypatch), test doubles (Mock, AsyncMock, fake) та аналіз statement і branch coverage.

1. Об'єкт тестування

Варіант 1. Система обліку студентів. Тестується застосунок `student_app`: студент із валідацією обов'язкових полів і списком оцінок, пошук за ID та прізвищем, фільтрація за групою, середній бал, найкращий студент, репозиторії (синхронний і асинхронний), сервіс сповіщень `Notifier`, CSV import студентів, JSON export звіту, YAML-конфігурація зі змінними середовища та потік «CSV → валідація → звіт».

Таблиця 1 – Структура production code (src/student_app)

Модуль	Відповідальність
`models.py`	`Student` (frozen dataclass): валідація ID, імен, групи й оцінок, `average`, `level`, `with_grade`
`grading.py`	`validate_grade` (0–100), `average_of`, `classify_grade` (4 рівні)
`repositories.py`	`StudentRepository` і `AsyncStudentRepository` (Protocol), in-memory реалізації
`services.py`	`StudentService`, `AsyncStudentService`, `Notifier` (Protocol): реєстрація, пошук, оцінки, сповіщення
`reports.py`	найкращий студент, середні по групах, розподіл за рівнями, звіт
`files.py`	CSV import із валідацією рядків, атомарний JSON export
`config.py`	YAML + змінні `STUDENT_APP_*`, перевірка
`app.py`	потік CSV → StudentService → JSON, CLI

2. Структура test suite і test matrix

Таблиця 2 – Файли тестів і кількість тестів (з урахуванням параметризації)

Файл	Тестів
`tests/integration/test_application_flow.py`	12
`tests/integration/test_async_flow.py`	1
`tests/integration/test_import_pipeline.py`	3
`tests/unit/test_async_service.py`	9
`tests/unit/test_configuration.py`	25
`tests/unit/test_csv_import.py`	27
`tests/unit/test_grading.py`	31
`tests/unit/test_json_export.py`	6
`tests/unit/test_models.py`	27
`tests/unit/test_reports.py`	13
`tests/unit/test_repositories.py`	9
`tests/unit/test_services.py`	32
Разом	**195**

Спільні fixtures винесено в `tests/conftest.py`, fake-реалізації – у `tests/fakes.py`. Integration tests позначено маркером `integration` (зареєстрований у `pyproject.toml`, `--strict-markers`), тому їх можна запускати окремо.

Таблиця 3 – Test matrix

Категорія	**Що перевіряється (приклади)**
Normal	реєстрація, пошук за ID та прізвищем, фільтр за групою, середній бал, найкращий студент, потік CSV → JSON
Min boundary	оцінка 0 і 0.01; середній бал 0 → unsatisfactory; один запис оцінок
Max boundary	оцінка 99.99 і 100; межі рівнів 59.99/60, 74.99/75, 89.99/90; середній бал рівно на порозі сповіщення (60)
Empty	порожній репозиторій (`NoStudentsError`), студент без оцінок, звіт для порожньої колекції, CSV лише із заголовком
Invalid type/value	оцінки −0.01, 100.01, NaN, ∞, текст, bool; ID 0, −5, bool, «1»; порожні й пробільні імена; некоректна група; 11 видів malformed CSV record
Missing data	немає config/ключів input_path, output_path; немає CSV, колонок; порожній файл; файл не в UTF-8
Dependency failure	Notifier → `NotificationError`, сховище → `RepositoryError`, async-сховище → `ConnectionError`, `json.dump` → `OSError`, `os.replace` → `PermissionError`, шлях виводу зайнятий файлом
Repeated operation	повторний student_id (в репозиторії та в CSV); повторний запуск потоку перезаписує звіт
State transition	невдале додавання оцінки не змінює студента й не надсилає сповіщення; збій сповіщення не скасовує реєстрацію; збій export не псує попередній файл

3. Fixtures

Використано autouse (`_clean_environment` прибирає змінні `STUDENT_APP_*`), scope=session (незмінний набір дозволених груп), fixture factory (`student_factory` з лічильником ID), autospec Mock (`notifier`), ланцюжок fixtures (`service` → `populated_service`) і fixture із yield `output_dir`, яка після тесту перевіряє, що атомарний export не лишив тимчасових файлів.

```
@pytest.fixture(autouse=True)
def _clean_environment(monkeypatch: pytest.MonkeyPatch) -> None:
    for name in ENV_NAMES:
        monkeypatch.delenv(ENV_PREFIX + name, raising=False)

@pytest.fixture(scope="session")
def allowed_groups() -> frozenset[str]:
    return frozenset({"PI-21", "PI-22", "PI-23"})

@pytest.fixture
def student_factory() -> StudentFactory:
    counter = itertools.count(1)

    def create(
        last_name: str = "Коваленко",
        first_name: str = "Олена",
        group: str = "PI-21",
        grades: tuple[float, ...] = (90.0,),
        student_id: int | None = None,
    ) -> Student:
        identifier = next(counter) if student_id is None else student_id
        return Student(identifier, first_name, last_name, group, grades)

    return create
```

```
@pytest.fixture
def notifier() -> Notifier:
    return create_autospec(Notifier, instance=True)

@pytest.fixture
def output_dir(tmp_path: Path) -> Iterator[Path]:
    directory = tmp_path / "output"
    yield directory
    leftovers = list(directory.glob("*.tmp")) if directory.exists() else []
    assert leftovers == [], f"тимчасові файли не видалено: {leftovers}"
```

4. Приклади unit tests

Parametrization і boundary test: межі рівнів оцінювання (з `ids` для читабельного виводу).

```
@pytest.mark.parametrize(
    ("average", "level"),
    [
        (0, "unsatisfactory"),
        (59.99, "unsatisfactory"),
        (60, "satisfactory"),
        (74.99, "satisfactory"),
        (75, "good"),
        (89.99, "good"),
        (90, "excellent"),
        (100, "excellent"),
    ],
    ids=[
        "min",
        "below-60",
        "exactly-60",
        "below-75",
        "exactly-75",
        "below-90",
        "exactly-90",
        "max",
    ],
)
def test_classify_grade_boundaries(average: float, level: str) -> None:
    assert classify_grade(average) == level
    assert level in LEVELS
```

pytest.raises, custom exception і pytest.approx: середній бал без оцінок піднімає `EmptyGradesError`; середнє від чисел із плаваючою комою порівнюється з допуском.

```
def test_average_without_grades_raises_custom_error(student_factory: StudentFactory) -> None:
    student = student_factory(grades=())
    with pytest.raises(EmptyGradesError, match="без оцінок") as error:
        _ = student.average
    assert isinstance(error.value, ValueError)

def test_average_tolerates_floating_point_noise(student_factory: StudentFactory) -> None:
    student = student_factory(grades=(0.1, 0.2))
    assert student.average != 0.15
    assert student.average == pytest.approx(0.15)
```

Mock, return_value, interaction assertion:

```
def test_register_saves_student_and_notifies_once(
    service: StudentService, notifier: Mock, student_factory: StudentFactory
) -> None:
    student = student_factory(group="PI-22")
    service.register(student)
    assert service.get(student.student_id) is student
    notifier.send.assert_called_once_with(student, "Вас зараховано до групи PI-22")

def test_service_reads_student_through_mock_repository(student_factory: StudentFactory) -> None:
    student = student_factory(student_id=7)
    repository = create_autospec(StudentRepository, instance=True)
    repository.get.return_value = student
    service = StudentService(repository, create_autospec(Notifier, instance=True))
    assert service.get(7) is student
    repository.get.assert_called_once_with(7)
```

side_effect і assert_not_called: збій сповіщення не скасовує реєстрацію, а збій сховища не допускає сповіщення.

```
def test_notifier_failure_does_not_roll_back_registration(
    service: StudentService,
    notifier: Mock,
    student_factory: StudentFactory,
    caplog: pytest.LogCaptureFixture,
) -> None:
    notifier.send.side_effect = NotificationError("SMTP недоступний")
    student = student_factory(student_id=6)
    with caplog.at_level(logging.WARNING):
        service.register(student)
    assert service.get(6) is student
    assert service.failed_notifications == [6]
    assert "Не вдалося сповістити студента 6" in caplog.text

def test_repository_failure_prevents_notification(
    student_factory: StudentFactory,
) -> None:
    repository = create_autospec(StudentRepository, instance=True)
    notifier = create_autospec(Notifier, instance=True)
    repository.add.side_effect = RepositoryError("сховище недоступне")
    service = StudentService(repository, notifier)
    with pytest.raises(RepositoryError):
        service.register(student_factory())
    notifier.send.assert_not_called()
    assert service.failed_notifications == []
```

monkeypatch і tmp_path: змінна середовища перевизначає YAML; підміна `json.dump` перевіряє, що атомарний export не псує попередній файл.

```
def test_environment_variables_override_yaml(
    config_writer: ConfigWriter, monkeypatch: pytest.MonkeyPatch
) -> None:
    path = config_writer(pass_threshold="60")
    monkeypatch.setenv("STUDENT_APP_PASS_THRESHOLD", "75")
    monkeypatch.setenv("STUDENT_APP_SKIP_INVALID", "no")
    monkeypatch.setenv("STUDENT_APP_OUTPUT_PATH", "elsewhere/result.json")
    config = load_config(path)
    assert config.pass_threshold == pytest.approx(75.0)
    assert config.skip_invalid is False
    assert config.output_path.name == "result.json"

def test_failed_write_keeps_previous_file_and_leaves_no_temporary_file(
    output_dir: Path, monkeypatch: pytest.MonkeyPatch
) -> None:
    target = output_dir / "report.json"
    write_json(target, {"version": 1})

    def broken_dump(*args: object, **kwargs: object) -> None:
        raise OSError("диск переповнено")

    monkeypatch.setattr(files.json, "dump", broken_dump)
    with pytest.raises(ExportError, match="report.json") as error:
        write_json(target, {"version": 2})
    assert isinstance(error.value.__cause__, OSError)
    assert json.loads(target.read_text(encoding="utf-8")) == {"version": 1}
```

5. AsyncMock

Асинхронний репозиторій – зовнішня залежність, тому `AsyncStudentService` тестується з `create_autospec(AsyncStudentRepository)`, який створює `AsyncMock` для кожного `async def`. Перевіряються результат, `assert_awaited_once_with`, `assert_not_awaited` і збій мережі (`side_effect=ConnectionError`), що перетворюється на `RepositoryError` зі збереженою причиною. Режим pytest-asyncio – `strict`.

```
@pytest.fixture
def async_repository() -> AsyncMock:
    return create_autospec(AsyncStudentRepository, instance=True)

async def test_load_returns_student_from_async_repository(
    async_service: AsyncStudentService, async_repository: AsyncMock, student_factory: StudentFactory
) -> None:
    student = student_factory(student_id=1)
    async_repository.get.return_value = student
    assert await async_service.load(1) is student
    async_repository.get.assert_awaited_once_with(1)

async def test_connection_failure_becomes_repository_error_with_cause(
    async_service: AsyncStudentService, async_repository: AsyncMock
) -> None:
    async_repository.get.side_effect = ConnectionError("таймаут")
    with pytest.raises(RepositoryError, match="сховище недоступне") as error:
        await async_service.load(1)
    assert isinstance(error.value.__cause__, ConnectionError)
```

6. Integration tests

Integration tests працюють із реальними файлами в `tmp_path` і справжніми компонентами (config → CSV import → валідація → StudentService → report → JSON export); замінено лише зовнішню межу – `Notifier`. Окремий async-тест поєднує імпорт із `AsyncStudentService`.

```
def test_pipeline_writes_json_report(
    csv_writer: CsvWriter, config_writer: ConfigWriter, output_dir: Path
) -> None:
    csv_writer(*ROWS)
    config = load_config(config_writer(allowed_groups="[PI-21, PI-22, PI-23]"))
    report = run_pipeline(config, create_autospec(Notifier, instance=True))
    saved = json.loads(config.output_path.read_text(encoding="utf-8"))
    assert saved == report
    assert saved["total_students"] == 5
    assert saved["graded_students"] == 4
    assert saved["best_student"]["student_id"] == 3
    assert saved["at_risk"] == [4]
    assert [e["line"] for e in saved["import_errors"]] == [7, 8]
    assert saved["import_errors"][1]["message"] == "група PI-24 не дозволена"
```

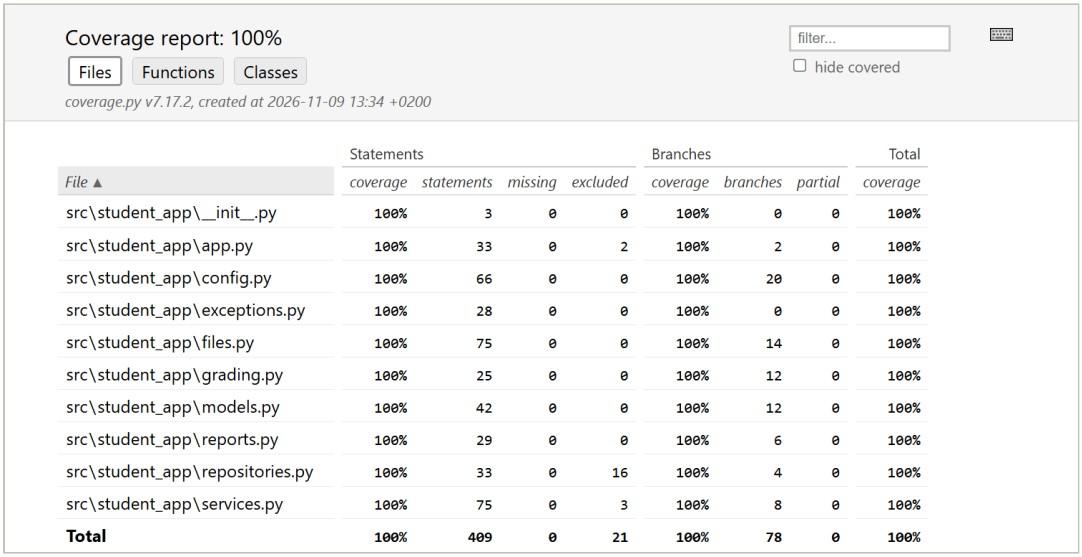
7. Результати запуску та coverage

```
PS C:\Github\prof_python\Lab_06> pytest -q --cov=student_app --cov-branch --cov-report=term-missing --cov-
report=html
.......................................................................... [ 36%]
.......................................................................... [ 73%]
....................................................                     [100%]
============================== tests coverage ==============================
_____________ coverage: platform win32, python 3.12.10-final-0 _____________

Name                                Stmts   Miss Branch BrPart  Cover   Missing
-------------------------------------------------------------------------------
src\student_app\__init__.py             3      0      0      0   100%
src\student_app\app.py                 33      0      2      0   100%
src\student_app\config.py              66      0     20      0   100%
src\student_app\exceptions.py          28      0      0      0   100%
src\student_app\files.py               75      0     14      0   100%
src\student_app\grading.py             25      0     12      0   100%
src\student_app\models.py              42      0     12      0   100%
src\student_app\reports.py             29      0      6      0   100%
src\student_app\repositories.py        33      0      4      0   100%
src\student_app\services.py            75      0      8      0   100%
-------------------------------------------------------------------------------
TOTAL                                 409      0     78      0   100%
Coverage HTML written to dir htmlcov
Required test coverage of 90.0% reached. Total coverage: 100.00%
195 passed in 0.88s
```

Рисунок 1 – Запуск усього suite: 195 passed, 0 failed, coverage 100.00 % (statement і branch)

Набір	**Результат**
`pytest -m "not integration"` (unit)	179 passed, 16 deselected in 0.41s
`pytest -m integration`	16 passed, 179 deselected in 0.32s



Coverage report: 100%

Files | Functions | Classes

filter... | hide covered

coverage.py v7.17.2, created at 2026-11-09 13:34 +0200

	Statements				Branches			Total
File ▲	coverage	statements	missing	excluded	coverage	branches	partial	coverage
src\student_app__init__.py	100%	3	0	0	100%	0	0	100%
src\student_app\app.py	100%	33	0	2	100%	2	0	100%
src\student_app\config.py	100%	66	0	0	100%	20	0	100%
src\student_app\exceptions.py	100%	28	0	0	100%	0	0	100%
src\student_app\files.py	100%	75	0	0	100%	14	0	100%
src\student_app\grading.py	100%	25	0	0	100%	12	0	100%
src\student_app\models.py	100%	42	0	0	100%	12	0	100%
src\student_app\reports.py	100%	29	0	0	100%	6	0	100%
src\student_app\repositories.py	100%	33	0	16	100%	4	0	100%
src\student_app\services.py	100%	75	0	3	100%	8	0	100%
Total	100%	409	0	21	100%	78	0	100%

Рисунок 2 – HTML coverage report (htmlcov/index.html), скриншот Microsoft Edge

Колонка «excluded» (21 рядків) – це блок `if __name__ == "__main__"` (правило `exclude_also` у pyproject.toml) і методи Protocol-інтерфейсів із тілом `...`, які coverage виключає за замовчуванням: виконуваного коду в них немає. Усі 78 гілок виконано в обох напрямках (колонка BrPart = 0).

8. Аналіз недостатньо протестованих paths і знайдені дефекти

Missing lines. Якщо запустити лише unit tests, загальне покриття падає до 95 %. Пропущені рядки показують, які шляхи перевіряють лише integration

tests:

```
PS C:\Github\prof_python\Lab_06> pytest -m "not integration" --cov=student_app --cov-branch
Name                                Stmts   Miss Branch BrPart  Cover   Missing
-------------------------------------------------------------------------------
src\student_app\__init__.py             3      0      0      0   100%
src\student_app\app.py                 33     20      2      0    37%   16-31, 35-49
src\student_app\config.py              66      0     20      0   100%
src\student_app\exceptions.py          28      0      0      0   100%
src\student_app\files.py               75      0     14      0   100%
src\student_app\grading.py             25      0     12      0   100%
src\student_app\models.py              42      0     12      0   100%
src\student_app\reports.py             29      0      6      0   100%
src\student_app\repositories.py        33      0      4      0   100%
src\student_app\services.py            75      2      8      0    98%   24, 53
-------------------------------------------------------------------------------
TOTAL                                 409     22     78      0    95%
```

Coverage for **src\student_app\app.py**: 37%

33 statements | 13 run | 20 missing | 2 excluded | 0 partial

« prev ^ index » next coverage.py v7.17.2, created at 2026-11-09 13:34 +0200

```
1  import argparse
2  import sys
3  from collections.abc import Sequence
4  from pathlib import Path
5  from typing import Any
6
7  from student_app.config import AppConfig, load_config
8  from student_app.exceptions import StudentAppError
9  from student_app.files import read_students_csv, write_json
10 from student_app.reports import build_report
11 from student_app.repositories import InMemoryStudentRepository
12 from student_app.services import LoggingNotifier, Notifier, StudentService
13
14
15 def run_pipeline(config: AppConfig, notifier: Notifier | None = None) -> dict[str, Any]:
16     imported = read_students_csv(
17         config.input_path,
18         skip_invalid=config.skip_invalid,
19         allowed_groups=config.allowed_groups,
20     )
21     service = StudentService(
22         InMemoryStudentRepository(), notifier or LoggingNotifier(), config.pass_threshold
23     )
24     for student in imported.students:
25         service.register(student)
26     report = build_report(service.all_students(), pass_threshold=config.pass_threshold)
27     report["import_errors"] = [
28         {"line": issue.line_number, "message": issue.message} for issue in imported.issues
29     ]
30     write_json(config.output_path, report)
31     return report
32
```

Рисунок 3 – Сторінка app.py у HTML coverage report після запуску лише unit tests: червоним позначено рядки, яких тести не виконали

Рядки 16–31 і 35–49 у `app.py` – функції `run_pipeline` і `main`; рядок 24 у `services.py` – `LoggingNotifier.send`; рядок 53 – `all_students`. Усі вони покриваються integration tests (повний потік, CLI), тож додавати нові unit tests немає потреби. Навпаки, лише integration tests дають 82 % і не перевіряють boundary та invalid-вхід, тому обидва види tests потрібні.

Coverage не гарантує відсутність дефектів. Перший варіант suite (186 tests) уже мав 100 % statement і branch coverage:

```
PS C:\Github\prof_python\Lab_06> pytest --cov=student_app --cov-branch --cov-report=term-missing   #
перший запуск
Name                              Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------------------------
src\student_app\__init__.py           3      0      0      0   100%
src\student_app\app.py               33      0      2      0   100%
src\student_app\config.py            66      0     20      0   100%
src\student_app\exceptions.py        28      0      0      0   100%
src\student_app\files.py             75      0     14      0   100%
src\student_app\grading.py           25      0     12      0   100%
src\student_app\models.py            41      0     10      0   100%
src\student_app\reports.py           29      0      6      0   100%
src\student_app\repositories.py      33      0      4      0   100%
src\student_app\services.py          75      0      8      0   100%
-----------------------------------------------------------------------------
TOTAL                               408      0     76      0   100%
Required test coverage of 90.0% reached. Total coverage: 100.00%
186 passed in 1.96s
```

Після цього були додані негативні тести на помилкові типи й порожні значення (9 tests), і вони виявили п'ять реальних дефектів production code:

```
PS C:\Github\prof_python\Lab_06> pytest   # після додавання негативних tests, до виправлень
FAILED tests/integration/test_application_flow.py::test_cli_reports_output_path_blocked_by_a_file
FAILED tests/unit/test_configuration.py::test_invalid_configuration_raises_configuration_error[input-null]
FAILED tests/unit/test_configuration.py::test_invalid_configuration_raises_configuration_error[output-
empty]
FAILED tests/unit/test_models.py::test_student_rejects_invalid_required_fields[group-none]
FAILED tests/unit/test_models.py::test_student_rejects_invalid_required_fields[group-number]
FAILED tests/unit/test_models.py::test_student_rejects_invalid_required_fields[group-unicode-digits]
FAILED tests/unit/test_models.py::test_student_rejects_grades_that_are_not_a_sequence[none]
FAILED tests/unit/test_models.py::test_student_rejects_grades_that_are_not_a_sequence[text]
FAILED tests/unit/test_models.py::test_student_rejects_grades_that_are_not_a_sequence[number]
9 failed, 186 passed in 0.37s
```

Таблиця 4 – Дефекти production code, знайдені tests

№	Симптом	Причина	Виправлення
D1	`Student(group=None)` і `group=21` → `AttributeError: 'NoneType' object has no attribute 'strip'` замість `InvalidStudentError`	група нормалізувалась до перевірки типу	перевірка `isinstance(group, str)` перед нормалізацією
D2	група `PI-٢١` (арабсько-індійські цифри) приймається (`DID NOT RAISE`)	у `str`-шаблоні `\d` збігається з будь-якою Unicode-цифрою	шаблон `[A-Z]{2}-[0-9]{2}`
D3	`grades=None` → `TypeError: 'NoneType' object is not iterable`; `grades="90"` → оцінка «9» і `InvalidGradeError` замість помилки типу	тип колекції оцінок не перевірявся	явна перевірка `tuple | list` → `InvalidStudentError`
D4	`input_path: (null)` і `output_path: ''` приймаються (`DID NOT RAISE ConfigurationError`)	`None` перетворювався на шлях «None»	порожнє значення вважається відсутнім ключем
D5	CLI, якщо каталог виводу зайнятий файлом: необроблений `FileExistsError` замість коду виходу 1	`mkdir` виконувався поза блоком `try`	`mkdir` перенесено в `try`, помилка стає `ExportError`

Після виправлень усі 195 tests проходять, а покриття лишилося 100 %. Висновок: відсоток coverage показує лише виконаність коду; дефекти знайшли tests із осмисленими assertions для invalid-входу.

9. Експерименти

Таблиця 5 – Експерименти 1–2: слабкий і сильний тест на «мутанті» (середнє замінено на максимум)

Тест	Код без помилки	Мутант	Coverage models.py
`test_average_weak` (без assert)	passed	passed	75.9 %
`test_average_strong`	passed	failed	75.9 %

Обидва тести виконують однакові рядки й дають однаковий coverage, але помилку ловить лише тест з assert: execution ≠ verification, а coverage показує лише те, що код виконувався.

Таблиця 6 – Експерименти 3–7

Експеримент	Результат
3. Branch coverage `classify_grade`	лише excellent і unsatisfactory: 4/6 гілок (66.7 %), пропущено переходи 29→30, 31→32 (рівні good і satisfactory); з усіма чотирма рівнями: 6/6 (100.0 %)
4. Mock vs fake	після еквівалентного рефакторингу (`StudentService.get` через `repository.all()` замість `repository.get()`): тест з Mock – failed, з fake – passed
5. Flaky test	випадкові ID студентів: 3 падінь із 20 запусків; з фіксованим seed: 0 із 20
6. tmp_path isolation	3 запуски – різні тимчасові каталоги; змінених файлів у репозиторії: 0
7. Failure path	success і failure залежностей: Notifier (`return` / `side_effect=NotificationError`), сховища (`RepositoryError`, `ConnectionError`), export (`OSError`); перевірено і виняток, і відсутність небажаних взаємодій (`assert_not_called`, `assert_not_awaited`)

Mock перевіряє взаємодію і тому прив'язаний до реалізації – ламається при рефакторингу без зміни поведінки; fake перевіряє стан, легше читається й перевикористовується, але потребує власного коду. Mock доречний на системних межах (сповіщення, мережа), fake – для сховищ. Flaky-тест небезпечний тим, що підриває довіру до червоних збірок; випадковість треба фіксувати (seed, ін'єкція даних).

Висновки

Створено test suite із 195 тестів для системи обліку студентів (варіант 1): unit tests моделі, оцінювання, репозиторіїв, сервісів, звітів, конфігурації, CSV import і JSON export, async-сервісу та integration tests повного потоку CSV → валідація → звіт, CLI і async-потоку. Використано fixtures різних видів, параметризацію, pytest.raises, approx, tmp_path, monkeypatch, Mock/autospec, AsyncMock і fake. Негативні тести знайшли п'ять дефектів production code, які не бачив 100 % coverage; statement і branch coverage – 100.00 % при порозі 90 % у pyproject.toml. Експерименти показали, що coverage не замінює змістовних assertions, а детермінованість та ізоляція тестів важливіші за їх кількість.